SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – High (Coding Challenge)

Course Code:	CSA0501	**Course Title:**	**Programming in Java**
CO Assessed:	CO1 – Precision (BL3,BL4)	**Assessment:**	Assessment Tool 1 – Coding Challenge
Weightage:	40% of CIA	**Total Marks:**	100 (5 Questions × 20 Marks)
CO1 Statement:	*Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*		
Student Name:	Pranav Adarsh G	**Reg. No.:**	1924260042
Section / Batch:	Slot-B	**Date:**	22.07.2026

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- *This test contains 10 questions, each carrying 10 marks. Total: 100 Marks.*
- *No negative marking. Unanswered questions carry zero marks.*
- *No calculators or electronic devices permitted.*
- *Duration: 90 minutes.*

ANNEXURE

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

Input:

```
public class Main {

    static class Book {

        private int bookId;

        private String title, author;

        private double price;

        private boolean issued;

        Book(int id, String t, String a, double p) {

            bookId = id;

            title = t;

            author = a;

            price = p;

            issued = false;

        }

        public int getBookId() {

            return bookId;

        }

        public String getTitle() {

            return title;

        }
```

```
    public void issueBook() {
        if (!issued) {
            issued = true;
            System.out.println(title + " Issued");
        } else {
            System.out.println("Already Issued");
        }
    }
    public void returnBook() {
        issued = false;
        System.out.println(title + " Returned");
    }
    public void display() {
        System.out.println(bookId + " " + title + " " + author + " " + price);
    }
}
public static void main(String[] args) {
    Book[] b = {
        new Book(101, "Java", "James", 500),
        new Book(102, "Python", "Guido", 450)
    };
    for (Book x : b)
        x.display();
    b[0].issueBook();
    b[0].returnBook();
```

```
        int searchId = 102;
        for (Book x : b)
            if (x.getBookId() == searchId)
                System.out.println("Book Found: " + x.getTitle());
    }
}
```

Ouput:

101 Java James 500.0

102 Python Guido 450.0

Java Issued

Java Returned

Book Found: Python

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

Input:

```
public class Main {
    static class Employee {
        String name;


        Employee(String n) {
            name = n;
        }
        void calculateSalary() {
            System.out.println("Salary");
        }
    }
    static class PermanentEmployee extends Employee {
        double basic;
        PermanentEmployee(String n, double b) {
            super(n);
            basic = b;
        }
        void calculateSalary() {
            System.out.println(name + " Salary = " + (basic + 5000));
        }
    }
    static class ContractEmployee extends Employee {
        int hours;
        double rate;
```

```
    ContractEmployee(String n, int h, double r) {
        super(n);
        hours = h;
        rate = r;
    }
    void calculateSalary() {
        System.out.println(name + " Salary = " + (hours * rate));
    }
  }
  public static void main(String[] args) {
    Employee e;
    e = new PermanentEmployee("Ravi", 30000);
    e.calculateSalary();
    e = new ContractEmployee("Kiran", 20, 500);
    e.calculateSalary();
  }
}
```

Output:

Ravi Salary = 35000.0

Kiran Salary = 10000.0

3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

Input:

```
public class Main {
    static class Vehicle {
        void calculateRent(int days) {
            System.out.println("Rent");
        }
    }
    static class Car extends Vehicle {
        void calculateRent(int days) {
            System.out.println("Car Rent = " + (days * 1000));
        }
    }
    static class Bike extends Vehicle {
        void calculateRent(int days) {
            System.out.println("Bike Rent = " + (days * 500));
        }
    }
    static class Bus extends Vehicle {
        void calculateRent(int days) {
            System.out.println("Bus Rent = " + (days * 2000));
        }
    }
    public static void main(String[] args) {
        Vehicle[] v = new Vehicle[3];
        v[0] = new Car();
```

```
        v[1] = new Bike();
        v[2] = new Bus();
        int days = 3;
        for (Vehicle x : v)
            x.calculateRent(days);
    }
}
```

Output:

```
Car Rent = 3000
Bike Rent = 1500
Bus Rent = 6000
```

4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

Input:

```
public class Main {
    static class BankAccount {
```

```
    private int accountNumber;
    private String holderName;
    private double balance;
    BankAccount(int accNo, String name, double bal) {
        accountNumber = accNo;
        holderName = name;
        balance = bal;
    }
    void deposit(double amount) {
        balance += amount;
    }
    void withdraw(double amount) {
        if (amount <= balance)
            balance -= amount;
        else
            System.out.println("Insufficient Balance");
    }
    void showBalance() {
        System.out.println("Account No : " + accountNumber);
        System.out.println("Holder Name: " + holderName);
        System.out.println("Balance    : " + balance);
    }
}
public static void main(String[] args) {
    BankAccount b = new BankAccount(101, "Ravi", 5000);
```

```
        b.deposit(2000);

        b.withdraw(1500);

        b.showBalance();

        b.withdraw(7000);   // Invalid transaction

    }

}
```

Output:

```
Account No : 101

Holder Name: Ravi

Balance    : 5500.0

Insufficient Balance
```

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

Input:

```
public class Main {
```

```
    static class Admission {
        void calculateFee() {
            System.out.println("Undergraduate Fee = 50000");
        }
        void calculateFee(int pg) {
            System.out.println("Postgraduate Fee = 70000");
        }
        void calculateFee(double scholarship) {
            System.out.println("Scholarship Fee = " + (50000 - scholarship));
        }
    }
    public static void main(String[] args) {
        Admission a = new Admission();
        a.calculateFee();          // Undergraduate
        a.calculateFee(1);         // Postgraduate
        a.calculateFee(20000.0);   // Scholarship
    }
}
```

Output:

Undergraduate Fee = 50000

Postgraduate Fee = 70000

Scholarship Fee = 30000.0

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

Input:

```
public class Main {

    static abstract class Shape {

        abstract void area();

    }

    static class Circle extends Shape {

        double r;

        Circle(double r) {

            this.r = r;

        }

        void area() {

            System.out.println("Circle Area = " + (3.14 * r * r));

        }

    }

    static class Rectangle extends Shape {

        double l, b;

        Rectangle(double l, double b) {

            this.l = l;

            this.b = b;

        }

        void area() {

            System.out.println("Rectangle Area = " + (l * b));
```

```
        }
    }
    static class Triangle extends Shape {
        double base, height;
        Triangle(double base, double height) {
            this.base = base;
            this.height = height;

        void area() {
            System.out.println("Triangle Area = " + (0.5 * base * height));
        }
    }
    public static void main(String[] args) {
        Shape[] s = {
            new Circle(5),
            new Rectangle(4, 6),
            new Triangle(8, 5)
        };
        for (Shape x : s)
            x.area();
    }
}
```

Output:

Circle Area = 78.5

Rectangle Area = 24.0

Triangle Area = 20.0

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

Input:

```
public class Main {

    interface MedicalRecord {

        void addRecord();

        void displayRecord();

    }
```

```
static class Patient implements MedicalRecord {
    String name = "Ravi";
    public void addRecord() {
        System.out.println("Patient Record Added");
    }
    public void displayRecord() {
        System.out.println("Patient Name: " + name);
    }
}
static class Doctor implements MedicalRecord {
    String name = "Dr. Kumar";
    public void addRecord() {
        System.out.println("Doctor Record Added");
    }
    public void displayRecord() {
        System.out.println("Doctor Name: " + name);
    }
}
public static void main(String[] args) {
    MedicalRecord p = new Patient();
    MedicalRecord d = new Doctor();
    p.addRecord();
    p.displayRecord();
    d.addRecord();
    d.displayRecord();
```

```
    }
}
```

Output:

Patient Record Added

Patient Name: Ravi

Doctor Record Added

Doctor Name: Dr. Kumar

8. Online Shopping Order System using Packages

Create two packages:

- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

Input:

```
public class Main {
    static class Product {
        String name;
        int price;
```

```
        Product(String n, int p) {
            name = n;
            price = p;
        }
    }
    static class Order {
        void bill(Product p, int qty) {
            System.out.println("Product : " + p.name);
            System.out.println("Total Amount : " + (p.price * qty));
        }
    }
    public static void main(String[] args) {
        Product p = new Product("Laptop", 50000);
        Order o = new Order();
        o.bill(p, 2);
    }
}
```

Input:

```
public class Main {
    static class Product {
        String name;
        int price;
        Product(String n, int p) {
            name = n;
            price = p;
```

```
        }
    }
    static class Order {
        void bill(Product p, int qty) {
            System.out.println("Product : " + p.name);
            System.out.println("Total Amount = " + (p.price * qty));
        }
    }
    public static void main(String[] args) {
        Product p = new Product("Laptop", 50000);
        Order o = new Order();
        o.bill(p, 2);
    }
}
```

Input:

```
public class Main {
    static class Product {
        String name;
        int price;
        Product(String n, int p) {
            name = n;
            price = p;
        }
    }
    static class Order {
```

```
    void bill(Product p, int qty) {
        System.out.println("Product : " + p.name);
        System.out.println("Total Amount = " + (p.price * qty));
    }
  }
  public static void main(String[] args) {

    Product p = new Product("Laptop", 50000);
    Order o = new Order();
    o.bill(p, 2);
  }
```

}**Output:**

Product : Laptop

Total Amount = 100000

9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

Input:

```
public class Main {
  static class Student {
    int rollNo, m1, m2, m3;
    String name;
    Student(int r, String n, int a, int b, int c) {
```

```
        rollNo = r;
        name = n;
        m1 = a;
        m2 = b;
        m3 = c;
    }
    int total() {
        return m1 + m2 + m3;
    }
    double average() {
        return total() / 3.0;
    }
    String grade() {
        if (average() >= 75)
            return "A";
        else if (average() >= 50)
            return "B";
        else
            return "C";
    }
}
public static void main(String[] args) {

    Student[] s = {
        new Student(1, "Ravi", 80, 75, 90),
```

```
            new Student(2, "Kiran", 70, 65, 60),
            new Student(3, "Rahul", 90, 95, 92)
        };
        Student topper = s[0];
        for (Student x : s) {
            System.out.println(x.rollNo + " " + x.name +
                    " Total=" + x.total() +
                    " Avg=" + x.average() +
                    " Grade=" + x.grade());

            if (x.total() > topper.total())
                topper = x;
        }
        System.out.println("Topper: " + topper.name);
    }
}
```

Output:

1 Ravi Total=245 Avg=81.66666666666667 Grade=A

2 Kiran Total=195 Avg=65.0 Grade=B

3 Rahul Total=277 Avg=92.33333333333333 Grade=A

Topper: Rahul

10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use

polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

Input:

```
public class Main {
    static class Device {
        void operate() {
            System.out.println("Device");
        }
    }
    static class Light extends Device {
        void operate() {
            System.out.println("Light ON - Power: 20W");
        }
    }
    static class Fan extends Device {
        void operate() {
            System.out.println("Fan ON - Power: 75W");
        }
    }
    static class AirConditioner extends Device {
        void operate() {
            System.out.println("Air Conditioner ON - Power: 1500W");
        }
    }
```

```
    public static void main(String[] args) {
        Device[] d = new Device[3];
        d[0] = new Light();
        d[1] = new Fan();
        d[2] = new AirConditioner();
        for (Device x : d)
            x.operate();
    }
}
```

Output:

Light ON - Power: 20W
Fan ON - Power: 75W
Air Conditioner ON - Power: 1500W

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks	
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints		
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach		
Code Implementation	20%	Writes correct, efficient, and	Code works with minor errors or	Partially correct code; fails for some	Code does not execute or gives		

		clean code that passes all constraints	inefficiencies	cases	incorrect results		
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints		
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases		

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____